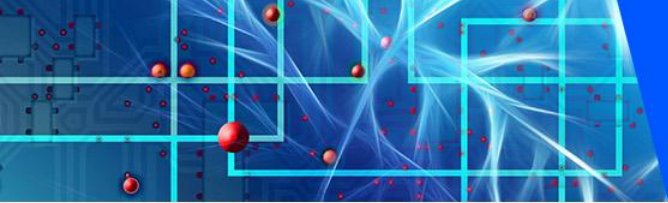


Presentación

- El aprendizaje no supervisado es una técnica de aprendizaje automático en la que el modelo analiza y encuentra patrones en datos sin etiquetas o respuestas predefinidas. Los algoritmos, como el clustering y la reducción de dimensionalidad, agrupan los datos o extraen características principales, permitiendo descubrir relaciones subyacentes, distribuciones y estructuras ocultas en los datos.



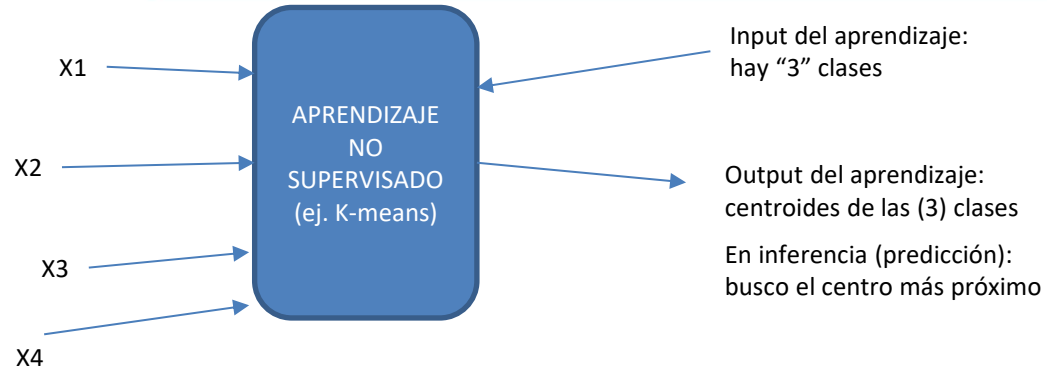
Algunos algoritmos

- K-means: Agrupa datos en clusters según proximidad.
- DBSCAN: Clustering basado en densidad.
- Algoritmo de agrupación jerárquica: Crea una jerarquía de clusters.
- Autoencoders: Redes neuronales para reducción de dimensionalidad.
- PCA (Análisis de Componentes Principales): Reduce dimensionalidad encontrando componentes principales.
- t-SNE: Visualización de datos en alta dimensionalidad.
- Apriori: Para reglas de asociación en minería de datos.
- Gaussian Mixture Model (GMM): Modela distribuciones de datos con múltiples gaussianas.
- Algoritmo de Expectation-Maximization (EM): Agrupa datos asumiendo distribuciones probabilísticas.
- Mapas Autoorganizados (SOM): Redes neuronales que agrupan datos y visualizan relaciones.

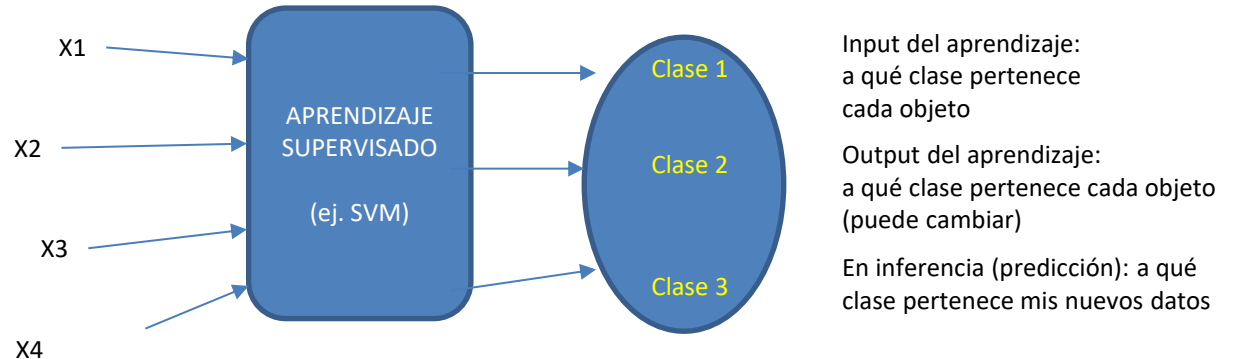
K-Means (I)

TABLAS DE DATOS
DE ENTRENAMIENTO

X1	X2	X3	X4
20	10	1	4
21	11	1	5
11	8	1	3
40	60	0	10
..	..	..	..



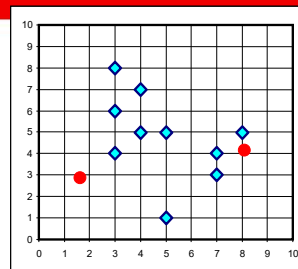
X1	X2	X3	X4	Y
20	10	1	4	Tu
21	11	1	5	Tu
11	8	1	3	Ar
40	60	0	10	To
..	..	..	..	..



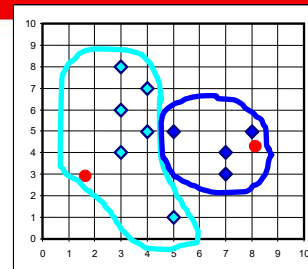
K-Means (II)

Método iterativo (sencillo)
que converge rápido

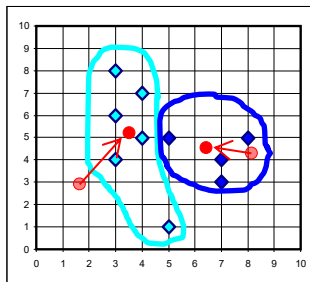
Partimos de 2
variables x, y



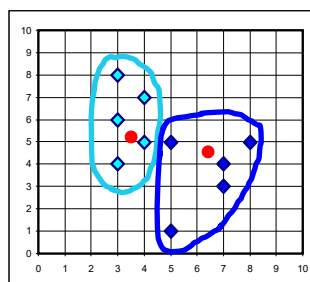
n semillas al azar(*)



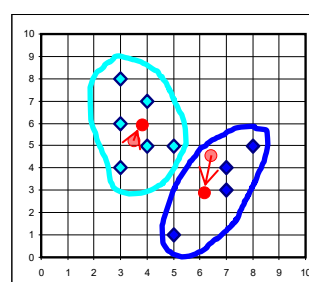
*más próximos
a cada semilla*



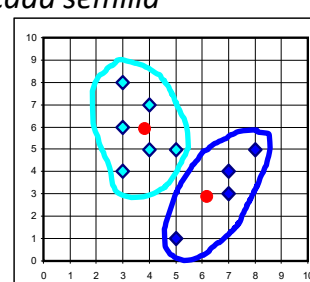
*calculo
centroide*



*más próximos
a cada semilla*



*calculo
centroide*



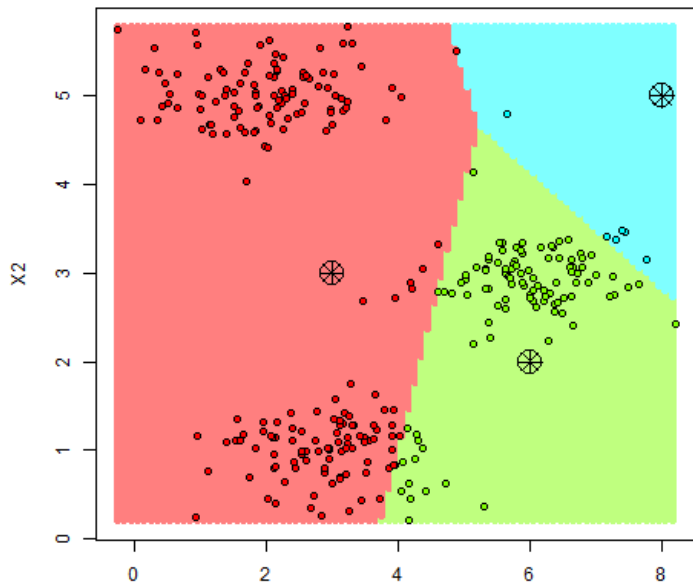
etc... hasta fin

(*) Nota: una elección “razonable” de las semillas facilita la convergencia

Tras el entrenamiento, se usa fácilmente para inferencia

K-Means (III)

Iteration number 1



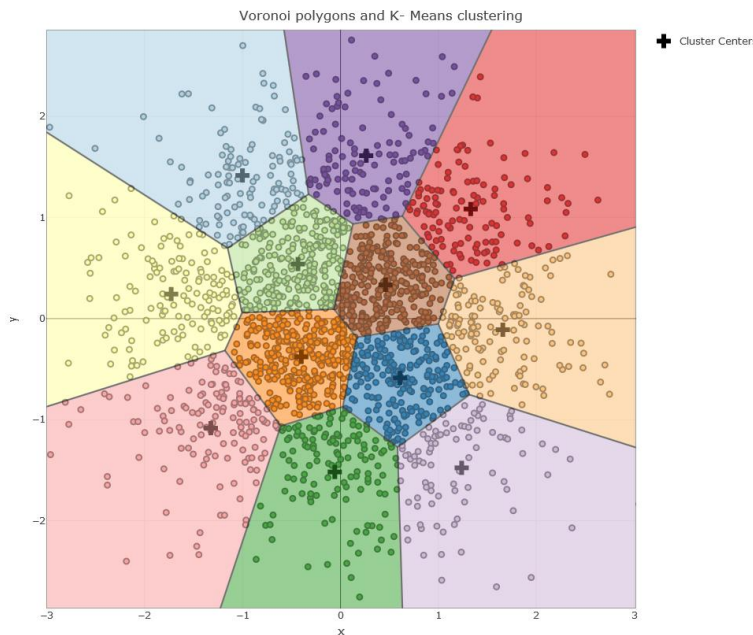
K-Means sólo sirve para regiones *globulares*.

Cuando se dice que un algoritmo de clustering "solo funciona para regiones globulares", significa que el algoritmo es efectivo para detectar y agrupar datos que forman clusters con formas aproximadamente esféricas o "globulares". En este caso, los puntos dentro de un cluster están más cerca unos de otros que de los puntos de otros clusters.

Por ejemplo, K-means es un algoritmo que tiende a funcionar bien para este tipo de regiones globulares porque basa sus agrupaciones en la proximidad de los puntos a los centroides (centros de los clusters). Sin embargo, si los clusters tienen formas más complejas o irregulares, como formas alargadas o en arco, K-means puede fallar al identificar correctamente los grupos, ya que asume que los clusters son esféricos.

Para clusters no globulares, algoritmos como DBSCAN o algoritmos jerárquicos pueden ser más efectivos.

K-Means (IV)



En general: n semillas, m dimensiones
(diagrama de Voronoi)

K-Means (VI)

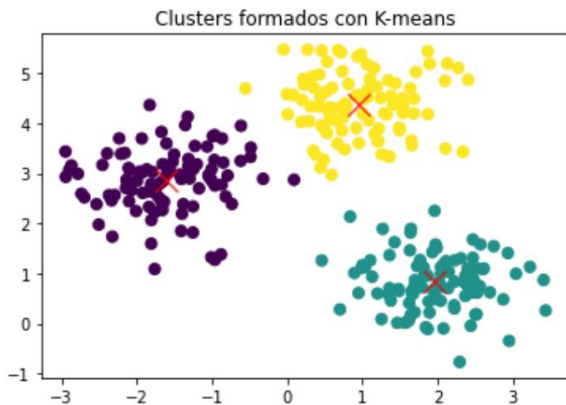
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

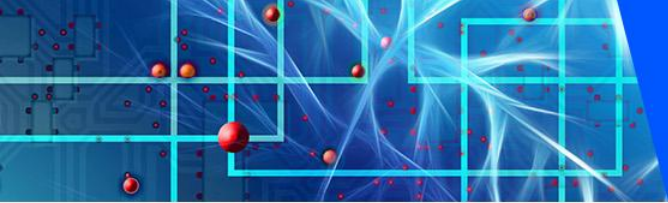
# Generamos un conjunto de datos de 300 puntos con 3 centros
X, y = make_blobs(n_samples=300, centers=3, cluster_std=0.60, random_state=0)
# Mostramos los datos generados
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title('Datos generados para el clustering')
plt.show()

# Creamos el modelo KMeans con k=3 (sabemos que los datos tienen 3 clusters)
kmeans = KMeans(n_clusters=3)
# Ajustamos el modelo a los datos
kmeans.fit(X)

# Obtenemos los centroides de los clusters y las etiquetas de cada punto
centroids = kmeans.cluster_centers_
etiquetas = kmeans.labels_

# Mostramos los resultados
plt.scatter(X[:, 0], X[:, 1], c=etiquetas, s=50, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', s=200, alpha=0.75, marker='x')
plt.title('Clusters formados con K-means')
plt.show()
```





Ejemplo K-Means Iris dataset (I)

```
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans
import pandas as pd
import seaborn as sns

# Cargamos el dataset Iris
iris = datasets.load_iris()
X = iris.data # Las características de las flores
y = iris.target # Las etiquetas verdaderas (aunque no las usamos en K-means)

# Creamos el modelo KMeans con 3 clusters
kmeans = KMeans(n_clusters=3, random_state=0)

# Ajustamos el modelo a los datos
kmeans.fit(X)
```


Ejemplo K-Means Iris dataset (II)

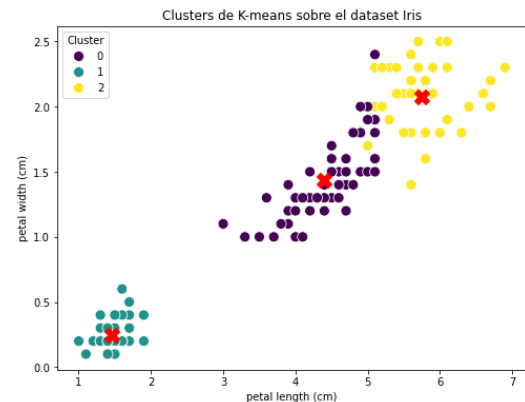
```
# Obtenemos las etiquetas predichas por K-means
etiquetas = kmeans.labels_

# Obtenemos los centroides de los clusters
centroides = kmeans.cluster_centers_

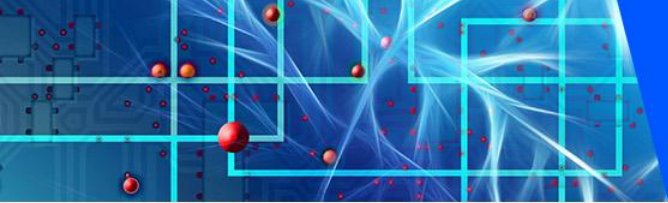
# Convertimos el dataset a un DataFrame para facilitar la visualización
df = pd.DataFrame(X, columns=iris.feature_names)
df['Cluster'] = etiquetas

# Usamos seaborn para hacer un scatter plot
plt.figure(figsize=(8,6))
sns.scatterplot(x=df['petal length (cm)'], y=df['petal width (cm)'],
hue=df['Cluster'], palette='viridis', s=100)

# Mostramos los centroides en el gráfico
plt.scatter(centroides[:, 2], centroides[:, 3], c='red', s=200, marker='X')
plt.title('Clusters de K-means sobre el dataset Iris')
plt.show()
```



Dado que el dataset tiene 4 características, podemos usar solo 2 características (ej., largo y ancho del pétalo) para una visualización más sencilla.



Ejercicio

Ejercicio: Detección de Anomalías en el Consumo de Energía de Edificios utilizando K-Means

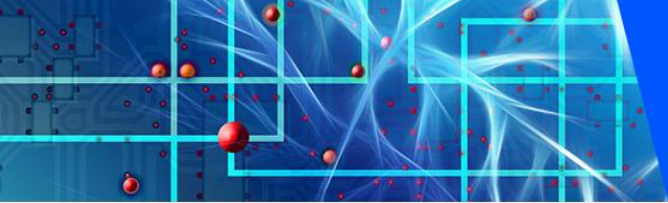
Contexto:

El consumo de energía de los edificios representa aproximadamente un tercio de la energía total a nivel mundial. Se estima que más del 20% de esta energía se desperdicia debido a fallos en los equipos, envejecimiento, errores de configuración y errores humanos. Este uso derrochador de energía puede ser identificado y prevenido mediante una técnica analítica basada en datos conocida como detección de anomalías o detección de valores atípicos. Los edificios modernos están equipados con medidores de energía inteligentes, que generan periódicamente una gran cantidad de datos en series temporales, alcanzando millones de puntos de datos cada día. Este conjunto de datos puede ser aprovechado para identificar anomalías en los perfiles de consumo de energía, lo que representa un paso importante hacia el ahorro de energía y la sostenibilidad global.

Objetivo:

Construir un sistema de detección de anomalías en los datos de consumo energético de un edificio utilizando el algoritmo de K-Means. El sistema debe identificar patrones de consumo inusuales que puedan indicar posibles fallos o comportamientos anómalos en los equipos o sistemas.

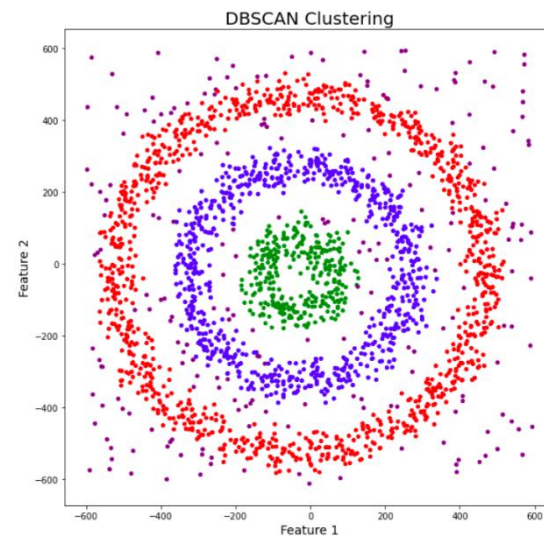
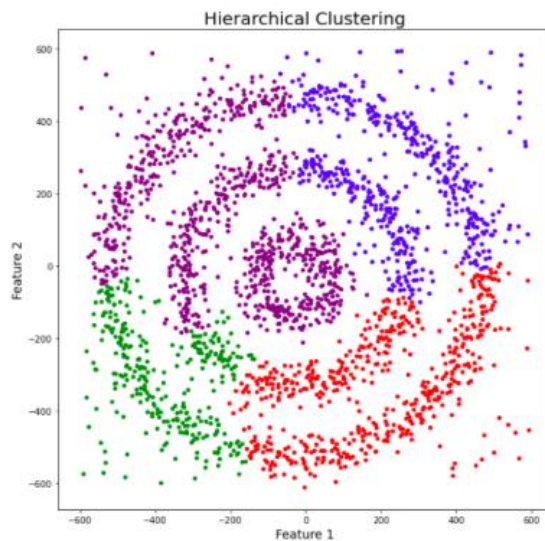
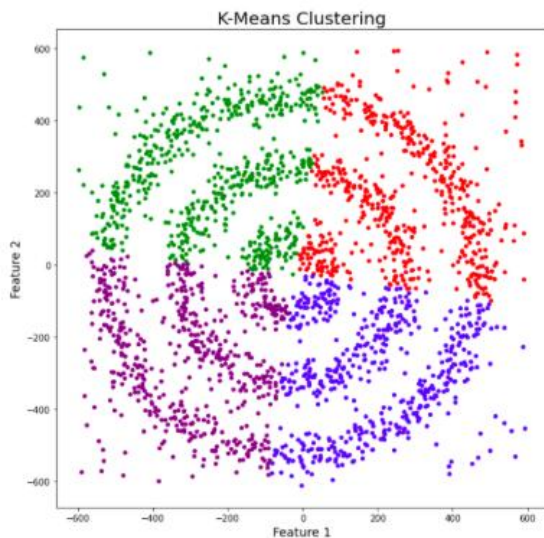
<https://www.kaggle.com/competitions/energy-anomaly-detection>



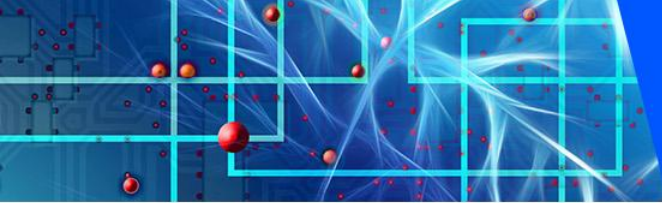
DBScan (I)

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise) es un algoritmo de clustering basado en densidad que agrupa puntos en función de la densidad de los datos. Es especialmente útil para encontrar clusters de forma arbitraria, no necesariamente esféricos, y también para identificar puntos de ruido (outliers). A diferencia de K-means, no requiere especificar el número de clusters por adelantado.

DBScan (II)



<https://www.analyticsvidhya.com/blog/2020/09/how-dbscan-clustering-works/>

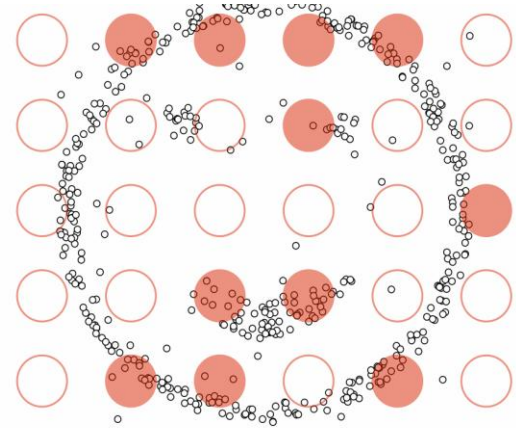


DBScan (III)

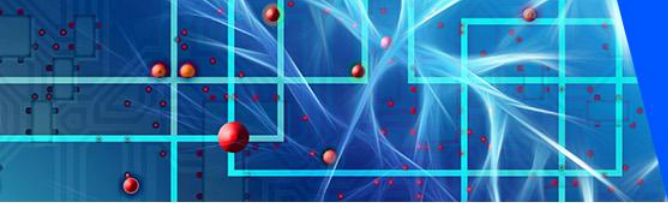
- Conceptos clave de DBSCAN:
 - Puntos de core (núcleo): Un punto es considerado de core si tiene al menos un número mínimo de puntos (`min_samples`) dentro de una distancia (epsilon, o `eps`). Estos puntos están en regiones densas.
 - Puntos frontera: Son puntos que están dentro de la distancia epsilon de un punto core, pero no tienen suficientes vecinos para ser considerados de core por sí mismos.
 - Puntos de ruido: Son puntos que no son ni puntos de core ni puntos frontera, es decir, están en áreas de baja densidad y no pertenecen a ningún cluster.

DBScan (IV)

- Funcionamiento del algoritmo:
 - Para cada punto:
 - Si tiene al menos min_samples dentro de su radio ϵ , se considera un punto de core y forma parte de un nuevo cluster.
 - Si no, se clasifica como punto frontera o punto de ruido.
 - Expandir clusters:
 - Los puntos de core extienden el cluster incorporando puntos cercanos (dentro de ϵ) que también son puntos de core o frontera.
 - Este proceso continúa hasta que no se encuentren más puntos que puedan ser agregados al cluster.
 - Asignar ruido:
 - Los puntos que no pertenecen a ningún cluster (ni son de core ni frontera) se clasifican como ruido o outliers.



<https://img-blog.csdnimg.cn/47a49a44c21e41559ea62fad71206bec.gif>



DBScan (V)

- Ventajas de DBSCAN:
 - No requiere especificar el número de clusters, ya que lo determina automáticamente en función de la densidad.
 - Puede encontrar clusters de cualquier forma, no solo esféricos (lo que es una limitación de K-means).
 - Es robusto a outliers, ya que los puntos dispersos se etiquetan como ruido.
- Desventajas:
 - Elegir los parámetros epsilon y min_samples de manera adecuada puede ser complicado.
 - No funciona bien si los datos tienen variaciones de densidad muy grandes, ya que un solo valor de epsilon podría no ser adecuado para todas las áreas.

DBScan (VI)

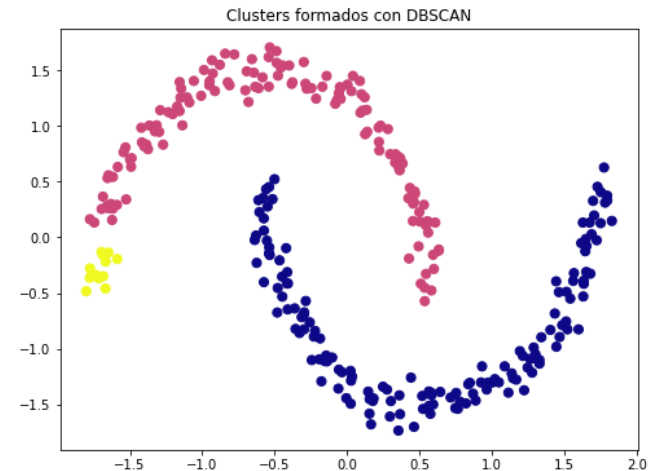
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

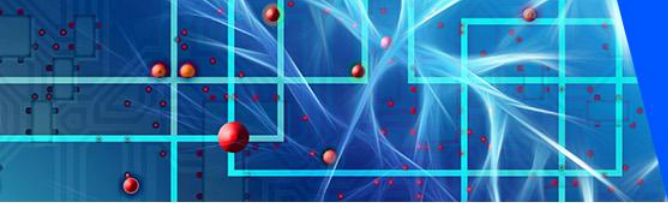
# Generamos datos de ejemplo con forma no esférica (arcos o lunas)
X, y = make_moons(n_samples=300, noise=0.05)

# Escalamos los datos para mejorar el rendimiento de DBSCAN
X = StandardScaler().fit_transform(X)

# Aplicamos DBSCAN con eps=0.2 y min_samples=5
dbscan = DBSCAN(eps=0.2, min_samples=5)
etiquetas = dbscan.fit_predict(X)

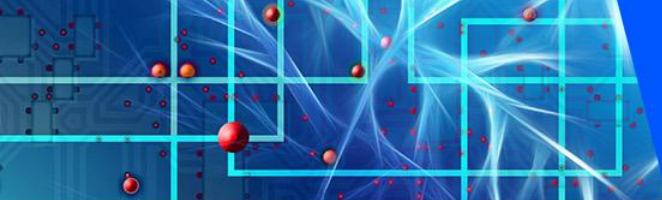
# Visualizamos los resultados
plt.figure(figsize=(8,6))
plt.scatter(X[:, 0], X[:, 1], c=etiquetas, cmap='plasma', s=50)
plt.title('Clusters formados con DBSCAN')
plt.show()
```





Análisis de componentes principales (I)

- PCA (Análisis de Componentes Principales) y clustering son técnicas que pueden complementarse bien cuando trabajamos con datos de alta dimensionalidad. PCA ayuda a reducir la dimensionalidad de los datos, facilitando su visualización y el rendimiento de los algoritmos de clustering, como K-means o DBSCAN.
- PCA (Principal Component Analysis) es una técnica de reducción de dimensionalidad que transforma los datos originales a un nuevo sistema de coordenadas, donde cada nueva dimensión (o componente principal) captura la mayor variabilidad posible en los datos. El objetivo es reducir el número de dimensiones, manteniendo tanta información como sea posible.
- ¿Por qué usar PCA en clustering?
 - Datos de alta dimensionalidad: Cuando los datos tienen muchas características (dimensiones), realizar clustering directamente sobre ellos puede ser ineficiente y difícil de interpretar.
 - Reducción de ruido: PCA ayuda a eliminar el ruido en los datos, conservando solo las características más importantes, lo que puede mejorar el rendimiento del clustering.
 - Visualización: En muchos casos, es útil reducir los datos a 2 o 3 dimensiones usando PCA para visualizar los clusters formados por algoritmos como K-means o DBSCAN.

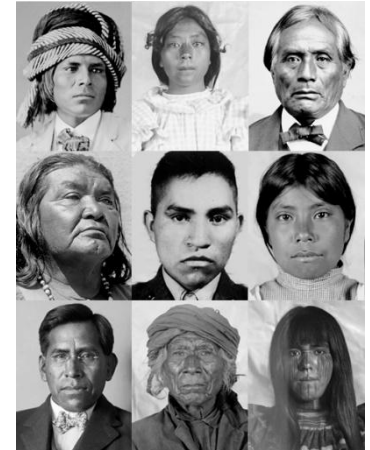


Análisis de componentes principales (II)

- El análisis de componentes principales (o PCA) utiliza el álgebra lineal para transformar el conjunto de datos en una forma comprimida.
- En general, se trata de una técnica de reducción de datos. Una propiedad del PCA es que se puede elegir el número de dimensiones o componentes principales en el resultado transformado.
- En el ejemplo siguiente, utilizamos PCA y seleccionamos 6 componentes principales.
- **OJO: PCA no indica inicialmente qué características son más importantes. PCA reduce el conjunto de features a un nuevo conjunto con menos elementos. En el ejemplo siguiente, reduciría el juego de 8 features a 6.**

Análisis de componentes principales (III)

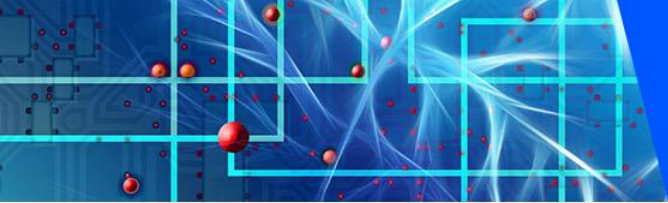
- **Título: Base de datos de la diabetes de los indios Pima**
 - La variable diagnóstica, de valor binario, investigada es si el paciente muestra signos de diabetes según los criterios de la Organización Mundial de la Salud (es decir, si la glucosa plasmática de 2 horas después de la carga era de al menos 200 mg/dl en cualquier examen de la encuesta o si se detectó durante la atención médica rutinaria). La población vive cerca de Phoenix, Arizona, Estados Unidos.
 - Número de muestras: 768
 - Número de atributos: 8 más la clase
 - Para cada atributo: (todos con valor numérico)
 1. Número de veces que está embarazada
 2. Concentración de glucosa en plasma a las 2 horas en una prueba de tolerancia a la glucosa oral
 3. Presión arterial diastólica (mm Hg)
 4. Espesor del pliegue cutáneo del tríceps (mm)
 5. Insulina sérica de 2 horas (μ U/ml)
 6. Índice de masa corporal ($\text{peso en kg}/(\text{altura en m})^2$)
 7. Función de pedigrí de la diabetes
 8. Edad (años)
 9. Variable de clase (0 o 1)



https://es.wikipedia.org/wiki/Pueblo_pima

Análisis de componentes principales (IV)

```
# Dimension reduction with PCA
import numpy
import pandas as pd
from pandas import read_csv
from sklearn.decomposition import PCA
# Visualization
import matplotlib.pyplot as plt
# load data
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = read_csv(url, names=names)
array = dataframe.values
X = array[:,0:8]
Y = array[:,8]
# feature extraction
n_pcs = 6
pca = PCA(n_components=n_pcs)
fit = pca.fit(X)
# summarize components
print("Explained Variance: %s" % fit.explained_variance_ratio_)
print(fit.components_)
# Ojo -> Seleccionar los componentes en un nuevo dataset.
X_pca = pca.transform(X)
```



Análisis de componentes principales (V)

```
# Calculate cumulative explained variance across all PCs
cum_exp_var = []
var_exp = 0
for i in pca.explained_variance_ratio_:
    var_exp += i
    cum_exp_var.append(var_exp)
# Plot cumulative explained variance for all PCs
fig, ax = plt.subplots(figsize=(8,6))
ax.bar(range(1,n_pcs + 1), cum_exp_var)
ax.set_xlabel('# Principal Components')
ax.set_ylabel('% Cumulative Variance Explained')
# get the index of the most important feature on EACH component i.e. largest absolute value using LIST
COMPREHENSION HERE
most_important = [numpy.abs(pca.components_[i]).argmax() for i in range(n_pcs)]
initial_feature_names = ['a','b','c','d','e']
# get the names
# using LIST COMPREHENSION HERE AGAIN
dic = {'PC{0}'.format(i+1): names[i] for i in range(n_pcs)}
# build the dataframe
df = pd.DataFrame(sorted(dic.items()))
print(df)
```

Análisis de componentes principales (VI)

Explained Variance: [0.889 0.062 0.026 0.013 0.007 0.003]

	preg	plas	pres	skin	test	mass	pedi	age
PC1	[-2.022e-03	9.781e-02	1.609e-02	6.076e-02	9.931e-01	1.401e-02	5.372e-04	-3.565e-03]
PC2	[-2.265e-02	-9.722e-01	-1.419e-01	5.786e-02	9.463e-02	-4.697e-02	-8.168e-04	-1.402e-01]
PC3	[-2.246e-02	1.434e-01	-9.225e-01	-3.070e-01	2.098e-02	-1.324e-01	-6.400e-04	-1.255e-01]
PC4	[-4.905e-02	1.198e-01	-2.627e-01	8.844e-01	-6.555e-02	1.928e-01	2.699e-03	-3.010e-01]
PC5	[1.516e-01	-8.794e-02	-2.322e-01	2.600e-01	-1.723e-04	2.147e-02	1.641e-03	9.205e-01]
PC6	[-5.047e-03	5.074e-02	7.564e-02	2.214e-01	-6.133e-03	-9.708e-01	-2.029e-03	-1.511e-02]

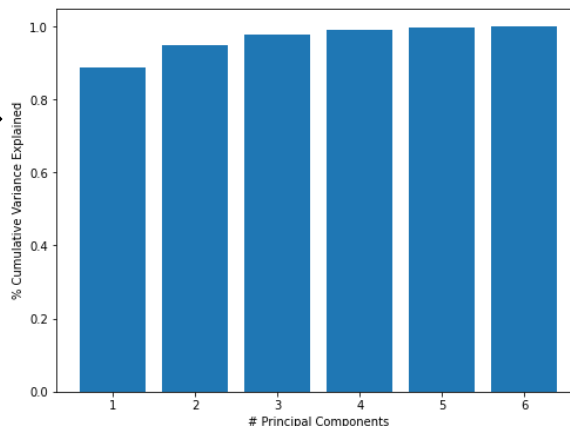
↑
Aportación de cada
feature a los 6 PC
seleccionados

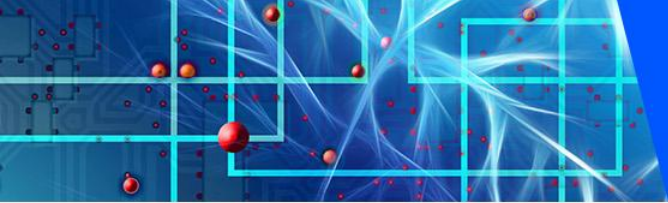
Feature más
utilizado para
producir cada
PC

→

	0	1
0	PC1	preg
1	PC2	plas
2	PC3	pres
3	PC4	skin
4	PC5	test
5	PC6	mass

¿Cuántos PC
son necesarios
para obtener un
resultado ideal
(100%)?
Podrían valer 3.

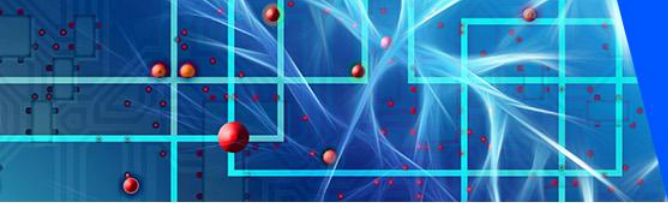




Análisis de componentes principales (VII)

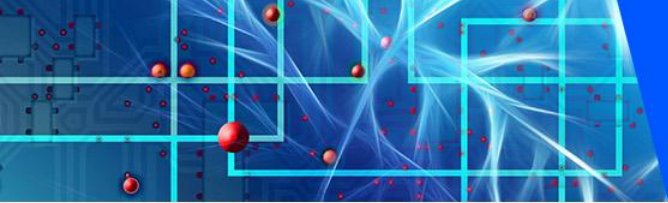
PCA tiene múltiples usos actualmente además de la reducción de dimensionalidad:

- Mejora del Rendimiento de Modelos: Reducir la dimensionalidad puede ayudar a mejorar el rendimiento de los modelos de machine learning, especialmente cuando se enfrenta a problemas de sobreajuste.
- Visualización de Datos: PCA se utiliza para proyectar datos en un espacio de menor dimensión (por ejemplo, de tres dimensiones a dos dimensiones) para facilitar la visualización y la interpretación de los datos. Esto es particularmente útil para conjuntos de datos complejos y de alta dimensionalidad.
- Eliminación de Ruido: Las componentes con menor varianza suelen corresponder a ruido o a variaciones no significativas.
- Compresión de Datos: PCA permite comprimir los datos en un espacio de menor dimensión, lo que es útil para el almacenamiento y la transmisión de datos. Los datos pueden ser reconstruidos posteriormente (aunque con alguna pérdida de información).
- Control de los modelos generativos



Selección de variables/características

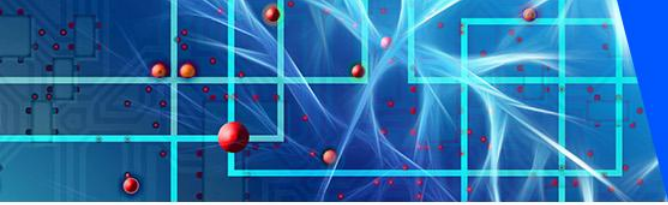
- La selección de características es un proceso en el que se seleccionan automáticamente las características de los datos que más contribuyen a la variable de predicción o al resultado en el que se está interesado.
- La presencia de características irrelevantes en los datos puede reducir la precisión de muchos modelos, especialmente de los algoritmos lineales como la regresión lineal y logística.



Beneficios

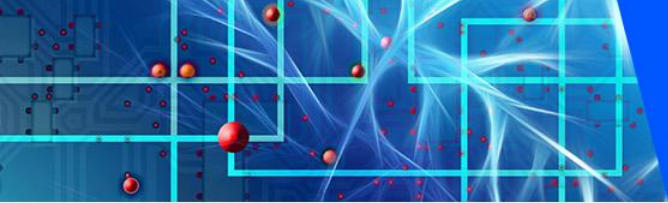
Los tres beneficios de realizar una selección de características antes de modelar los datos son:

1. Reduce el *overfitting* (sobreajuste): Menos datos redundantes significa menos oportunidades de tomar decisiones basadas en el ruido.
2. Mejora la precisión: Menos datos erróneos significa que la precisión del modelado mejora.
3. Reduce el tiempo de entrenamiento: Menos datos significa que los algoritmos se entrenan más rápido.



Tipos de algoritmos

1. Selección univariante
2. Eliminación recursiva de características
3. Análisis de componentes principales (visto anteriormente)
4. Importancia de las características



Selección univariante (I)

- Las pruebas estadísticas se pueden utilizar para seleccionar las características que tienen la relación más fuerte con la variable de salida.
- La biblioteca scikit-learn proporciona la clase `SelectKBest` que puede ser utilizada con un conjunto de diferentes pruebas estadísticas para seleccionar un número específico de características.
- Se pueden utilizar muchas pruebas estadísticas diferentes con este método de selección. Por ejemplo, el método del valor F de ANOVA es apropiado para entradas numéricas y datos categóricos, como vemos en el conjunto de datos Pima.
- Se puede utilizar mediante la función `f_classif()`. Seleccionaremos las 4 mejores características utilizando este método en el ejemplo siguiente.

Selección univariante (II)

Para calcular el valor F, se sigue este proceso:

1. Calcular la variabilidad entre grupos (SST):

La suma de cuadrados entre los grupos (SST) mide la variabilidad debida a las diferencias entre las medias de los grupos.

$$SST = \sum_{i=1}^k n_i (\bar{y}_i - \bar{y})^2$$

Donde:

- k es el número de grupos.
- n_i es el tamaño del grupo i .
- $\bar{y}_i$ es la media del grupo i .
- $\bar{y}$ es la media global (de todos los datos).

2. Calcular la variabilidad dentro de los grupos (SSE):

La suma de cuadrados dentro de los grupos (SSE) mide la variabilidad que no se puede explicar por las diferencias entre las medias de los grupos, es decir, la variabilidad dentro de los grupos.

$$SSE = \sum_{i=1}^k \sum_{j=1}^{n_i} (y_{ij} - \bar{y}_i)^2$$

Donde:

- y_{ij} es el valor de la observación j en el grupo i .

Selección univariante (III)

3. Calcular los grados de libertad:

- Los grados de libertad entre grupos (df_{entre}):

$$df_{\text{entre}} = k - 1$$

- Los grados de libertad dentro de los grupos (df_{dentro}):

$$df_{\text{dentro}} = N - k$$

Donde N es el número total de observaciones.

4. Calcular las medias cuadráticas:

- La media cuadrática entre grupos (MST):

$$MST = \frac{SST}{df_{\text{entre}}}$$

- La media cuadrática dentro de los grupos (MSE):

$$MSE = \frac{SSE}{df_{\text{dentro}}}$$

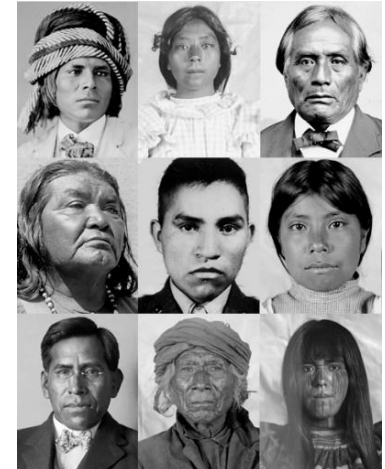
5. Calcular el valor F:

El valor F se calcula dividiendo la media cuadrática entre grupos por la media cuadrática dentro de los grupos:

$$F = \frac{MST}{MSE}$$

Selección univariante (IV)

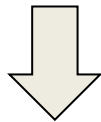
- **Título: Base de datos de la diabetes de los indios Pima**
 - La variable diagnóstica, de valor binario, investigada es si el paciente muestra signos de diabetes según los criterios de la Organización Mundial de la Salud (es decir, si la glucosa plasmática de 2 horas después de la carga era de al menos 200 mg/dl en cualquier examen de la encuesta o si se detectó durante la atención médica rutinaria). La población vive cerca de Phoenix, Arizona, Estados Unidos.
 - Número de muestras: 768
 - Número de atributos: 8 más la clase
 - Para cada atributo: (todos con valor numérico)
 1. Número de veces que está embarazada
 2. Concentración de glucosa en plasma a las 2 horas en una prueba de tolerancia a la glucosa oral
 3. Presión arterial diastólica (mm Hg)
 4. Espesor del pliegue cutáneo del tríceps (mm)
 5. Insulina sérica de 2 horas (μ U/ml)
 6. Índice de masa corporal (peso en kg/(altura en m)²)
 7. Función de pedigrí de la diabetes
 8. Edad (años)
 9. Variable de clase (0 o 1)



Selección univariante (V)

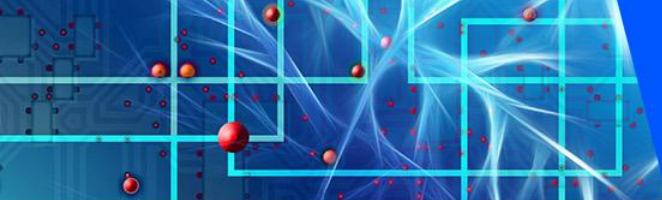
```
# Feature Selection with Univariate Statistical Tests
from pandas import read_csv
from numpy import set_printoptions
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_classif
# load data
```

```
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = read_csv(url, names=names)
array = dataframe.values
X = array[:,0:8]
Y = array[:,8]
# feature extraction
test = SelectKBest(score_func=f_classif, k=4)
fit = test.fit(X, Y)
# summarize scores
set_printoptions(precision=3)
print(fit.scores_)
```



Las 4 features que más contribuyen: 0 (preg), 1 (plas), 5 (mass) y 7 (age)

```
[ 39.67 213.162  3.257  4.304 13.281 71.772 23.871 46.141]
```

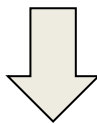


Eliminación recursiva de características (I)

- La Eliminación Recursiva de Características (o RFE) funciona eliminando recursivamente atributos y construyendo un modelo sobre los atributos que quedan.
- Utiliza la precisión del modelo para identificar qué atributos (y combinación de atributos) contribuyen más a la predicción del atributo objetivo.
- El ejemplo siguiente utiliza RFE con el algoritmo de regresión logística para seleccionar las 3 características principales. La elección del algoritmo no importa demasiado mientras sea hábil y consistente.

Eliminación recursiva de características (II)

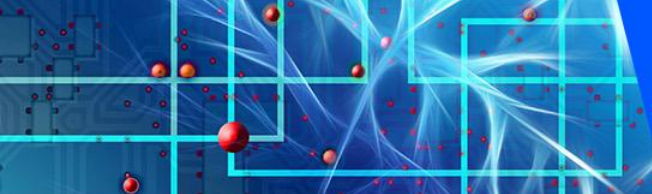
```
# Feature Extraction with RFE
from pandas import read_csv
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
# load data
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = read_csv(url, names=names)
array = dataframe.values
X = array[:,0:8]
Y = array[:,8]
# feature extraction
model = LogisticRegression(solver='lbfgs')
rfe = RFE(model, 3)
fit = rfe.fit(X, Y)
print("Num Features: %d" % fit.n_features_)
print("Selected Features: %s" % fit.support_)
print("Feature Ranking: %s" % fit.ranking_)
```



Las 3 features que más
contribuyen: 0 (preg), 6 (pedi) y
7 (age)

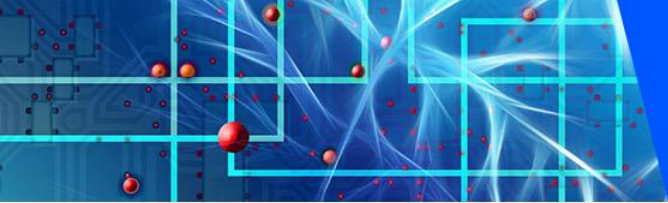
```
Num Features: 3
Selected Features: [ True False False False  True  True False]
Feature Ranking: [1 2 4 5 6 1 1 3]
```





Eliminación recursiva de características (III)

- ¿Por qué obtengo resultados diferentes entre distintos métodos e incluso dentro del mismo método?
 - Los resultados pueden variar debido a la naturaleza estocástica del algoritmo o del procedimiento de evaluación, o a las diferencias en la precisión numérica.
 - **Es necesario ejecutar el ejemplo varias veces y comparar el resultado medio.**

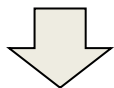


Importancia de las características (I)

- Los árboles de decisión, como Random Forest y Extra Trees, pueden utilizarse para estimar la importancia de las características.
 - Random Forest: es una combinación de árboles predictores tal que cada árbol depende de los valores de un vector aleatorio probado independientemente y con la misma distribución para cada uno de estos. Cada árbol es construido usando el siguiente algoritmo:
 - Sea N el número de casos de prueba, **M es el número de variables en el clasificador.**
 - Sea m el número de variables de entrada a ser usado para determinar la decisión en un nodo dado; **m debe ser mucho menor que M**
 - Elegir un conjunto de entrenamiento para este árbol y usar el resto de los casos de prueba para estimar el error.
 - **Para cada nodo del árbol, elegir aleatoriamente m variables en las cuales basar la decisión.** Calcular la mejor partición del conjunto de entrenamiento a partir de las m variables.
 - Extra Trees es como Random Forest, en el sentido de que construye múltiples árboles y divide los nodos utilizando subconjuntos aleatorios de características, pero con dos diferencias clave: No hace bootstrap de las observaciones (lo que significa que muestrea sin reemplazo), y los nodos se dividen en divisiones aleatorias, no en las mejores divisiones.
- En el ejemplo siguiente construimos un clasificador ExtraTreesClassifier para el conjunto de datos de los indios Pima sobre la aparición de la diabetes.

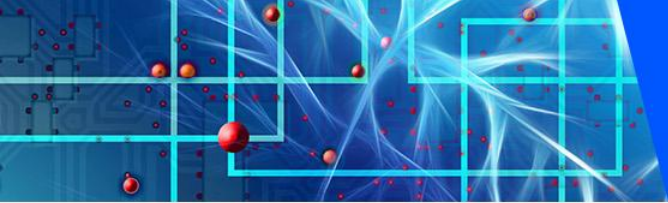
Importancia de las características (II)

```
# Feature Importance with Extra Trees Classifier
from pandas import read_csv
from sklearn.ensemble import ExtraTreesClassifier
# load data
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = read_csv(url, names=names)
array = dataframe.values
X = array[:,0:8]
Y = array[:,8]
# feature extraction
model = ExtraTreesClassifier(n_estimators=10)
model.fit(X, Y)
print(model.feature_importances_)
```



plas pres mass pedi age

[0.095 0.222 0.111 0.081 0.08 0.141 0.124 0.146]

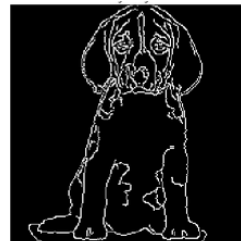
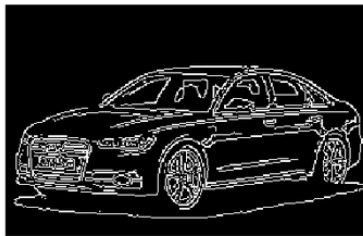


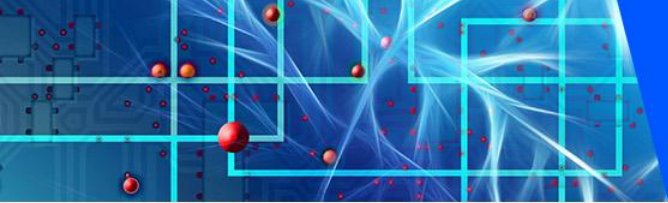
Más sobre los árboles de decisión

- Decision Trees/Árbol de decisión (alta varianza)
 - Un árbol de decisión único **suele ajustarse en exceso a los datos de los que aprende** porque sólo aprende de una vía de decisiones. Las predicciones de un único árbol de decisión no suelen ser precisas en los nuevos datos.
- Random Forest/Bosque aleatorio (Varianza media)
 - Los modelos de bosque **aleatorio reducen el riesgo de sobreajuste al introducir la aleatoriedad**
- Extra Trees/Árboles extra (baja varianza)
 - Como Random Forest, construyen múltiples árboles y divide los nodos utilizando subconjuntos aleatorios de características, pero no hace bootstrap de las observaciones (lo que significa que muestrea sin reemplazo), y **los nodos se dividen en divisiones aleatorias, no en las mejores divisiones.**

...en visión artificial

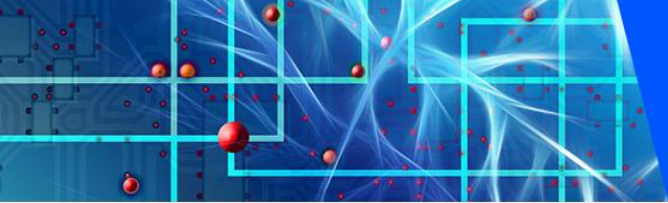
- En visión artificial existen técnicas que transformando la imagen de origen mediante filtros de convolución y otras técnicas permiten mantener el conocimiento necesario para resolver un problema concreto.
 - Se trata de una representación simplificada de la imagen que contiene sólo la información más importante de la misma.
 - Ejemplo: ¿Podemos diferenciar las dos imágenes en los siguientes casos?





Ejercicio 1

- Aplicar el algoritmo de K-Means para la clasificación del dataset "Wine", disponible en la biblioteca sklearn.
- Comparar los clusters obtenidos con las clases reales del dataset. Para ello reduce visualiza los clústeres en dos dimensiones utilizando PCA
- Muestra una matriz de confusión de las clases devueltas por K-Means y DBScan. ¿Hay algo extraño?
- Calcula el accuracy. ¿Hay algo extraño?



Ejercicio 2

- Aplicar el algoritmo de DBScan para la clasificación del dataset “wine”.
- Ajusta los parámetros `eps` y `min_samples` para obtener 3 clústeres
- Comparar los clusters obtenidos con las clases reales del dataset. Para ello reduce visualiza los clústeres en dos dimensiones utilizando PCA
- Muestra una matriz de confusión de las clases devueltas por DBScan. ¿Hay algo extraño?
- Calcula el accuracy. ¿Hay algo extraño?